Lecture 6. Generalized Linear Models and Model Evaluation

Logistic and Poisson regression, prediction error, cross-validation

Contents

6.1 Generalized linear models **2**

6.2 Logistic regression **2**

6.2.1 Coefficients as odds ratios 3

6.3 GLMs for counts **3**

6.3.1 Poisson regression 3

6.3.2 Overdispersion and the negative binomial 4

6.3.3 Zero-inflated Poisson 5

6.4 Evaluating models **5**

6.4.1 Training error and the holdout method 6

6.4.2 Cross-validation 6

6.4.3 Why a separate test set 7

6.1 Generalized linear models

Motivation. Linear regression assumes a continuous outcome that can take any value. Many outcomes are binary (diabetes or not), counts (articles published), or otherwise restricted. Generalized linear models keep the linear predictor but connect it to the outcome's mean through a link function suited to its type.

So far outcomes were quantitative. For a binary outcome, an outcome with several levels, or a count, we still use a linear predictor but through a monotone **link function** f:

$$f(\theta_i) = \beta_0 + \beta_1 x_{i1} + \cdots + \beta_p x_{ip}, \qquad y_i \sim D(\theta_i, \psi),$$

where θ_i is the centre of the distribution D and ψ controls its scale or shape.

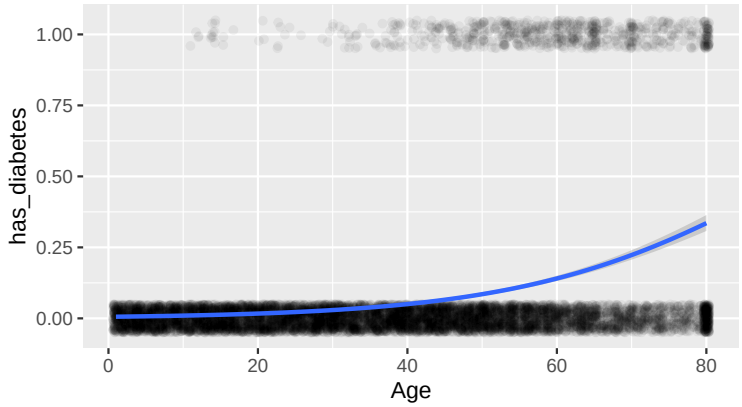
6.2 Logistic regression

For binary $y \sim \text{Bernoulli}(\theta)$ the link is the **logit**:

$$\text{logit}(\theta) = \log \frac{\theta}{1-\theta} = \beta_0 + \beta_1 x_1 + \cdots + \beta_p x_p, \qquad \theta = \frac{1}{1 + \exp\{-(\beta_0 + \beta_1 x_1 + \cdots + \beta_p x_p)\}} \in (0, 1).$$

For example, the probability of diabetes as a function of age and BMI in `NHANES`:

```
NHANES <- NHANES %>% mutate(has_diabetes = as.numeric(Diabetes == "Yes"))
NHANES %>% filter(!is.na(has_diabetes)) %>%
  ggplot(aes(Age, has_diabetes)) +
  geom_jitter(height = 0.05, alpha = 0.05) +
  geom_smooth(method = "glm", method.args = list(family = "binomial"), formula = y ~ x)
```



The probability rises with age (and similarly with BMI). Which matters more? A multiple logistic regression fits both, with `glm(..., family = "binomial")`:

```
logreg <- glm(has_diabetes ~ BMI + Age, family = "binomial", data = NHANES)
tidy(logreg)
```

```
## # A tibble: 3 x 5
##   term        estimate std.error statistic   p.value
##   <chr>          <dbl>     <dbl>     <dbl>     <dbl>
## 1 (Intercept)  -8.08     0.244       -33.1 1.30e-239
## 2 BMI           0.0943   0.00552      17.1 1.74e- 65
## 3 Age           0.0573   0.00249      23.0 2.28e-117
```

Both p-values are tiny. The predicted probability for a 60-year-old with BMI 25:

```r
linear_component <- c(1, 25, 60) %*% coef(logreg)
exp(linear_component) / (1 + exp(linear_component))
```

```
##            [,1]
## [1,] 0.09233095
```

```r
predict(logreg, newdata = data.frame(BMI = 25, Age = 60), type = "response")
```

```
##          1
## 0.09233095
```

6.2.1 Coefficients as odds ratios

Because the curve is nonlinear, a unit change in x does not change the probability by a constant amount. Coefficients are interpreted through **odds**, $p/(1-p)$, and **odds ratios**: if the odds of getting a job are $p/(1-p)$ with an MBA and $q/(1-q)$ without, their ratio is the factor by which an MBA changes the odds. With a single 0/1 predictor, $\operatorname{logit} P(y_i = 1) = \beta_0 + \beta_1 x_i$, the log odds are $\beta_0 + \beta_1$ when $x_i = 1$ and β_0 when $x_i = 0$, so with $p = P(y = 1 \mid x = 1)$ and $q = P(y = 1 \mid x = 0)$,

$$e^{\beta_1} = \frac{p/(1-p)}{q/(1-q)}.$$

In general e^{β_j} is the factor by which the odds are multiplied when x_j increases by one, other predictors fixed:

```r
exp(coef(logreg))
```

```
##  (Intercept)          BMI          Age
## 0.0003095805 1.0989173575 1.0589497173
```

Each additional year of age multiplies the odds of diabetes by about 1.06, and each BMI unit by about 1.10.

6.3 GLMs for counts

Motivation. Counts are non-negative integers whose variance usually grows with the mean. The Poisson model captures this in the simplest way; when the data are more variable or have extra zeros, the negative binomial and zero-inflated models extend it.

Count data record how many times something happened (car accidents per day, doctor visits per year). We consider Poisson, negative binomial and zero-inflated Poisson regression.

6.3.1 Poisson regression

A Poisson variable with rate λ has $P(x = k \mid \lambda) = e^{-\lambda}\lambda^k/k!$ for $k = 0, 1, 2, \ldots$, with mean and variance both λ; it models the number of events in a fixed interval (emails per hour). Poisson regression assumes $y_i \sim \text{Poisson}(\lambda_i)$ with the log link,

$$\log \lambda_i = \beta_0 + \beta_1 x_{i1} + \cdots + \beta_p x_{ip},$$

and no extra parameter ψ. e^{β_j} is the multiplicative effect of a one-unit increase in x_j on the rate: $\lambda(x_j + 1) = \lambda(x_j)\, e^{\beta_j}$.

Example. `pscl::bioChemists` records, for 915 biochemistry PhD students, the number of articles in the last three years of the PhD (`art`), gender, marital status, number of children under 5, departmental prestige and the mentor's article count.

```
bioChemists <- bioChemists %>% mutate(gender = ifelse(fem == "Men", 1, 0))
pois_mod <- glm(art ~ gender, family = "poisson", data = bioChemists)
round(coef(pois_mod), 3)
```

```
## (Intercept)      gender
##       0.385       0.247
```

```
exp(coef(pois_mod))
```

```
## (Intercept)      gender
##    1.470309    1.280405
```

For women ($x = 0$) $\hat{\lambda} = e^{0.385} = 1.47$ articles; for men $\hat{\lambda} = e^{0.385+0.247} = 1.88$. Men publish on average $e^{0.247} = 1.28$ times as many articles. (The slides write $\lambda_{\text{women}} = e^{0.247}$; the women's rate is $e^{\hat{\beta}_0} = e^{0.385}$, which is the value 1.470 given there [Supplement].)

6.3.2 Overdispersion and the negative binomial

If the variance exceeds what the model implies, the data are **overdispersed**. For counts this means the variance is much larger than the mean; fitting a Poisson model is then a misspecification that usually *underestimates* standard errors.

```
var(bioChemists$art) / mean(bioChemists$art)
```

```
## [1] 2.191358
```

The variance is more than twice the mean. The **negative binomial** distribution counts the failures before the r-th success in independent trials with success probability θ:

$$P(x = k \mid r, \theta) = \binom{r + k - 1}{k} \theta^r (1 - \theta)^k = \frac{\Gamma(r + k)}{\Gamma(r)\, k!} \theta^r (1 - \theta)^k,$$

the Gamma form allowing non-integer r. In R, `dnbinom(2, 3, 0.75)` is $P(x = 2 \mid r = 3, \theta = 0.75)$. Its mean is $\mu = r(1 - \theta)/\theta$, so $\theta = r/(r + \mu)$ and the distribution can be parameterised by μ and the dispersion r; the variance $\mu + \mu^2/r$ exceeds the mean.

```
dnbinom(2, 3, 0.75)
```

```
## [1] 0.1582031
```

A negative binomial is a mixture of Poissons with gamma-distributed means, so it fits data arising from many Poisson processes with different rates.

Poisson–gamma mixture [Supplement]

If $\lambda \sim \text{Gamma}(\text{shape } r,\ \text{rate } r/\mu)$ and $x \mid \lambda \sim \text{Poisson}(\lambda)$, then

$$P(x = k) = \int_0^\infty \frac{e^{-\lambda}\lambda^k}{k!} \frac{(r/\mu)^r \lambda^{r-1} e^{-r\lambda/\mu}}{\Gamma(r)}\, d\lambda = \frac{\Gamma(r + k)}{\Gamma(r)\, k!} \left(\frac{r}{r + \mu}\right)^r \left(\frac{\mu}{r + \mu}\right)^k,$$

the negative binomial with $\theta = r/(r + \mu)$. The extra variance μ^2/r is the variance of the gamma-distributed rates.

Negative binomial regression assumes $y_i \sim \text{NegBin}(\mu_i, r)$ with $\log \mu_i = \beta_0 + \beta x_i$; `MASS::glm.nb()` fits it:

```
nb_mod <- glm.nb(art ~ gender, data = bioChemists)
tidy(nb_mod)
```

```
## # A tibble: 2 x 5
##   term        estimate std.error statistic  p.value
##   <chr>          <dbl>     <dbl>     <dbl>    <dbl>
```

```
## 1 (Intercept)    0.385    0.0544      7.08 1.44e-12
## 2 gender         0.247    0.0720      3.43 6.01e- 4
```

```
tidy(pois_mod)
```

```
## # A tibble: 2 x 5
##   term        estimate std.error statistic  p.value
##   <chr>          <dbl>     <dbl>     <dbl>    <dbl>
## 1 (Intercept)    0.385    0.0402      9.59 8.77e-22
## 2 gender         0.247    0.0519      4.77 1.89e- 6
```

The coefficients agree, but the negative binomial standard errors are larger: the Poisson model was overconfident.

6.3.3 Zero-inflated Poisson

Some counts have more zeros than a Poisson allows, e.g. the number of cigarettes smoked, where non-smokers always report 0. A **zero-inflated Poisson** (ZIP) model is a mixture: with probability θ_i the unit is a "structural zero" ($z_i = 1$), otherwise its count is Poisson,

$$y_i \sim \theta_i \mathbb{1}(y_i = 0) + (1 - \theta_i)\,\text{Poisson}(\lambda_i), \qquad \log \lambda_i = \beta_0 + \beta_1 x_i, \quad \text{logit}\,\theta_i = \gamma_0 + \gamma_1 x_i,$$

a Poisson regression for the counts and a logistic regression for the zeros. The slides fit cigarettes smoked against years of education (`zeroinfl(cigs ~ educ)`) and find that more educated people are more likely to be non-smokers ($\hat{\theta}$ rises from 0.49 at 6 years to 0.72 at 18) but, if they smoke, smoke more ($\hat{\lambda}$ from 18.3 to 27.7). With `bioChemists` (many students published nothing):

```
zip_mod <- zeroinfl(art ~ gender | gender, data = bioChemists)
zip_mod
```

```
##
## Call:
## zeroinfl(formula = art ~ gender | gender, data = bioChemists)
##
## Count model coefficients (poisson with log link):
## (Intercept)       gender
##      0.6151       0.2393
##
## Zero-inflation model coefficients (binomial with logit link):
## (Intercept)       gender
##    -1.35416     -0.03902
```

The count part says men publish more when they publish; the zero part estimates how gender affects the chance of being a "non-publisher".

6.4 Evaluating models

Motivation. With many possible models we need a way to choose. The fit to the data used for estimation is optimistic, because flexible models also fit the noise; honest evaluation needs data the model has not seen.

Predictive modelling looks for a function g with $y = g(x) + \epsilon$: in linear regression $g(x) = \beta_0 + \beta_1 x_1 + \cdots + \beta_p x_p$, in a GLM $g(x) = f^{-1}(\beta_0 + \cdots + \beta_p x_p)$. To compare specifications we look at bias and variance.

A coin example. To estimate p for a fair coin without data: model A picks a uniform random number (zero bias, large variance); model B always says 0.7 (zero variance, large bias). Neither is good; models generally trade bias against variance.

Proposition 6.1 (bias–variance decomposition)

For a new point (x, y) with $y = g(x) + \epsilon$, $\mathbb{E}\epsilon = 0$, and ϵ independent of the fitted model $\hat{y}(\cdot)$,

$$\mathbb{E}\big[(y - \hat{y}(x))^2\big] = \mathbb{E}[\epsilon^2] + \big(g(x) - \mathbb{E}[\hat{y}(x)]\big)^2 + \mathbb{E}\big[(\mathbb{E}[\hat{y}(x)] - \hat{y}(x))^2\big].$$

Proof [Supplement]. Write $y - \hat{y} = \epsilon + \{g(x) - \mathbb{E}\hat{y}\} + \{\mathbb{E}\hat{y} - \hat{y}\}$. Square and take expectations: the middle term is a constant; $\mathbb{E}\epsilon = 0$ and independence kill the cross terms involving ϵ; and $\mathbb{E}\{\mathbb{E}\hat{y} - \hat{y}\} = 0$ kills the cross term between the last two. ■

Model risk = observation variance + (model bias)^2 + model variance.

- *Observation variance* σ^2: irreducible noise (measurement error, missing information), beyond the analyst's control.
- *Model bias*: the gap between the average prediction and the truth, from a family too limited (a quadratic fitted to cubic data), i.e. **underfitting**; reduced by more complexity or domain knowledge.
- *Model variance*: the fitted model would change with a different sample; large when small differences in samples lead to large differences in fits, i.e. **overfitting**; reduced by less complexity.

As complexity increases, bias falls and variance rises, so the risk first decreases and then increases.

6.4.1 Training error and the holdout method

Measuring risk on the data used for fitting (training error) always favours more complex models. The simplest remedy is the **holdout method**: set aside part of the data. For `RailTrail`, hold out every fifth day (18 of 90) and fit polynomials of degree 1–5 on the other 72:

```
index <- (1:18) * 5
test <- RailTrail[index, ]
train <- RailTrail[-index, ]
mods <- map(1:5, ~ glm(volume ~ poly(hightemp, .x), data = train))
tibble(degree = 1:5,
       train_mse = map_dbl(mods, ~ mean((train$volume - predict(.x))^2)),
       valid_mse = map_dbl(mods, ~ mean((test$volume - predict(.x, newdata = test))^2)),
       AIC = map_dbl(mods, AIC))
```

```
## # A tibble: 5 x 4
##   degree train_mse valid_mse   AIC
##    <int>     <dbl>     <dbl> <dbl>
## 1      1    11429.     7835.  883.
## 2      2    10277.     6531.  877.
## 3      3     9318.     4975.  872.
## 4      4     9210.     5128.  874.
## 5      5     9203.     5023.  875.
```

Training error keeps falling with degree; validation error is smallest for degree 3, which AIC also selects (though by a different principle: a penalty instead of new data). Strictly, the held-out part is a *validation* set, not a test set. Common splits are 80/20, 90/10 or 75/25; there is no general rule.

6.4.2 Cross-validation

A single split wastes data and depends on the split. In k**-fold cross-validation** the data are divided at random into k folds; each fold in turn is the validation set for a model fitted on the other

$k-1$, giving $\text{MSE}_1, \ldots, \text{MSE}_k$, and

$$\text{CV}_{(k)} = \frac{1}{k}\sum_{i=1}^{k} \text{MSE}_i.$$

With $k = 2$: fit on one half, test on the other, then swap; a model that does well in-sample but poorly out-of-sample is overfit. **Leave-one-out** CV ($k = n$) has no randomness and less bias than the holdout method, but needs n fits. `boot::cv.glm()` computes it:

```
set.seed(1)
cv_error <- map_dbl(1:5, function(i) {
  fit <- glm(volume ~ poly(hightemp, i), data = RailTrail)
  boot::cv.glm(RailTrail, fit, K = 5)$delta[1]
})
cv_error
```

```
## [1] 11386.289  9845.353  8797.032  9319.328  9336.853
```

LOOCV without refitting [Supplement]

For least squares, leaving out observation i changes its residual to $e_i/(1-h_{ii})$, with h_{ii} the leverage, so $\text{CV}_{(n)} = \frac{1}{n}\sum_i \{e_i/(1-h_{ii})\}^2$ needs a single fit.

```
fit <- lm(volume ~ poly(hightemp, 3), data = RailTrail)
mean((residuals(fit) / (1 - hatvalues(fit)))^2)
```

```
## [1] 8992.758
```

```
boot::cv.glm(RailTrail, glm(volume ~ poly(hightemp, 3), data = RailTrail))$delta[1]
```

```
## [1] 8992.758
```

6.4.3 Why a separate test set

After comparing many models on the validation set, the best validation loss is an optimistically biased estimate of the chosen model's performance. Analogy: among millions of equally good golfers, the tournament winner was probably also a bit lucky; to estimate the winner's true distance, have them hit once more. Likewise, report performance on a **test set** that was never used for fitting or selection. The ideal workflow: training set to fit, validation set (or CV) to select, test set once at the end.